

ui.scroll_area 全面详解

`ui.scroll_area` 是 NiceGUI 框架中基于 Quasar ScrollArea 组件封装的滚动区域组件，核心作用是为用户提供可自定义样式的滚动条，支持滚动事件监听、程序化控制滚动位置等功能，适用于需要限定显示区域并允许内容滚动的场景（如长文本展示、多元素滚动浏览等）。

一、核心功能与基础用法

1. 基础特性

- 封装 Quasar ScrollArea 组件，支持自定义滚动条样式（通过 `classes` `style` 等属性）；
- 限定内容显示区域，超出区域的内容自动生成滚动条；
- 支持垂直 / 水平滚动控制、滚动事件监听、程序化滚动定位。

2. 最小示例

通过 `with ui.scroll_area()` 包裹需要滚动的内容，需指定组件尺寸（如宽高）和边框样式（便于可视化区域范围）：

```
from nicegui import ui

with ui.row():
    # 带滚动功能的区域（宽32rem、高32rem、带边框）
    with ui.scroll_area().classes('w-32 h-32 border'):
        ui.label('I scroll. ' * 20) # 内容超出区域，会触发滚动

    # 无滚动功能的普通列（仅作为对比）
    with ui.column().classes('p-4 w-32 h-32 border'):
        ui.label('I will not scroll. ' * 10)

ui.run()
```

二、滚动事件处理 (on_scroll)

通过 `on_scroll` 参数绑定回调函数，可监听滚动位置变化，回调函数接收 `ScrollEventArguments` 对象，包含以下关键属性：

- `sender`：触发事件的滚动区域实例；
- `client`：对应的客户端实例；
- 其他属性（如 `vertical_percentage` 垂直滚动百分比、`horizontal_percentage` 水平滚动百分比）参考 Quasar ScrollArea API。

示例：实时显示滚动位置

```
from nicegui import ui

# 用于显示滚动位置的只读数字输入框
position = ui.number('scroll position:').props('readonly')

with ui.card().classes('w-32 h-32'):
    # 绑定滚动事件，实时更新滚动百分比
    with ui.scroll_area(on_scroll=lambda e: position.set_value(e.vertical_percentage)):
        ui.label('I scroll. ' * 20)

ui.run()
```

三、程序化控制滚动位置（scroll_to 方法）

通过 `scroll_to()` 方法可手动设置滚动位置，支持像素或百分比定位，适用于导航跳转、多滚动区域同步等场景。

方法参数说明

参数名	类型	说明
<code>pixels</code>	<code>Optional[float]</code>	滚动距离（像素，从顶部 / 左侧开始计算）
<code>percent</code>	<code>Optional[float]</code>	滚动百分比（0-1，如 0.5 表示中间位置）
<code>axis</code>	<code>Literal['vertical', 'horizontal']</code>	滚动轴（默认垂直 <code>vertical</code> ）
<code>duration</code>	<code>float</code>	滚动动画时长（秒，默认 0 无动画）

示例：多滚动区域同步 + 手动控制

```
from nicegui import ui

# 手动控制滚动位置的输入框（0-1 范围，步长 0.1）
ui.number('position', value=0, min=0, max=1, step=0.1,
         on_change=lambda e: area1.scroll_to(percent=e.value)).classes('w-32')

with ui.row():
    # 滚动区域1：滚动时同步到区域2
    with ui.card().classes('w-32 h-48'):
        with ui.scroll_area(on_scroll=lambda e:
            area2.scroll_to(percent=e.vertical_percentage)) as area1:
            ui.label('I scroll. ' * 20)

    # 滚动区域2：接收区域1的同步滚动
    with ui.card().classes('w-32 h-48'):
        with ui.scroll_area() as area2:
            ui.label('I scroll. ' * 20)

ui.run()
```

四、核心属性

属性名	类型	说明	
<code>classes</code>	<code>Classes[Self]</code>	组件的 CSS 类（如尺寸、边框、滚动条样式）	
<code>client</code>	<code>Client</code>	组件所属的客户端实例	
<code>html_id</code>	<code>str</code>	HTML DOM 中的元素 ID（2.16.0+ 版本支持）	
<code>is_deleted</code>	<code>bool</code>	组件是否已被删除	
<code>is_ignoring_events</code>	<code>bool</code>	组件是否忽略事件	
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	父插槽（可设置）
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 特性（如滚动条样式）	
<code>style</code>	<code>Style[Self]</code>	组件的内联 CSS 样式	
<code>visible</code>	<code>BindableProperty</code>	组件可见性（支持双向绑定）	

示例：自定义滚动条样式

通过 `classes` 结合 Tailwind CSS 或 Quasar 类修改滚动条样式：

```
from nicegui import ui

# 自定义滚动条（窄滚动条、hover 时变宽）
with ui.scroll_area().classes('w-64 h-48 border scrollbar-thin hover:scrollbar-wide'):
    ui.label('Custom scrollbar. ' * 30)

ui.run()
```

五、关键方法（常用）

除 `scroll_to()` 外，以下方法为高频使用场景：

1. 事件绑定相关

方法名	说明	参数要点
<code>on_scroll()</code>	绑定滚动事件（等价于初始化时的 <code>on_scroll</code> 参数）	接收回调函数，参数为 <code>ScrollEventArguments</code>
<code>on()</code>	订阅通用事件（如点击、鼠标事件）	<code>type</code> 为事件名（如 "click"）， <code>handler</code> 为回调

2. 组件控制相关

方法名	说明	参数要点
<code>clear()</code>	移除所有子元素	无参数
<code>delete()</code>	删除组件及所有子元素	无参数
<code>set_visibility()</code>	设置组件可见性	<code>visible: bool</code> (True 显示, False 隐藏)
<code>tooltip()</code>	为组件添加 tooltip 提示	<code>text: str</code> 为提示文本
<code>update()</code>	同步组件状态到客户端	无参数 (修改属性后需调用生效)

3. 资源与插槽相关

方法名	说明	参数要点
<code>add_slot()</code>	为组件添加 Vue 插槽 (复杂组件扩展用)	<code>name</code> 为插槽名, <code>template</code> 为 Vue 模板
<code>add_resource()</code>	添加资源 (如 CSS/JS 文件)	<code>path</code> 为资源路径

示例：动态控制组件可见性

```
from nicegui import ui

with ui.row():
    # 控制滚动区域可见性的开关
    toggle = ui.switch('Show Scroll Area', value=True)

    # 滚动区域：绑定开关状态
    with ui.scroll_area().bind_visibility_from(toggle, 'value').classes('w-32 h-32 border'):
        ui.label('Controlled visibility. ' * 20)

ui.run()
```

六、高级用法场景

1. 水平滚动

通过 `axis='horizontal'` 控制水平滚动，需确保内容横向超出区域：

```

from nicegui import ui

# 水平滚动区域（宽64rem、高16rem）
with ui.scroll_area().classes('w-64 h-16 border'):
    with ui.row(): # 横向排列内容
        for i in range(20):
            ui.button(f'Btn {i}', size='sm').classes('mr-2')

ui.run()

```

2. 滚动动画

通过 `duration` 参数设置滚动动画时长：

```

from nicegui import ui

with ui.scroll_area() as area:
    ui.label('Animated scroll. ' * 50)

# 点击按钮，1秒内滚动到顶部（0%）
ui.button('Scroll to Top', on_click=lambda: area.scroll_to(percent=0, duration=1.0))
# 点击按钮，1秒内滚动到底部（100%）
ui.button('Scroll to Bottom', on_click=lambda: area.scroll_to(percent=1.0, duration=1.0))

ui.run()

```

3. 多滚动区域联动

结合 `on_scroll` 和 `scroll_to` 实现多个滚动区域同步滚动：

```

from nicegui import ui

with ui.column():
    # 三个同步滚动的区域
    for i in range(3):
        with ui.scroll_area(on_scroll=lambda e, idx=i: sync_scroll(e, idx)).classes('w-64 h-24 border my-2') as area:
            ui.label(f'Area {i+1}: ' + 'Scroll sync. ' * 20)
            # 存储所有滚动区域实例
            if 'areas' not in locals():
                areas = []
            areas.append(area)

# 同步滚动逻辑
def sync_scroll(event, source_idx):
    percent = event.vertical_percentage
    for idx, area in enumerate(areas):
        if idx != source_idx: # 避免循环触发
            area.scroll_to(percent=percent)

ui.run()

```

七、版本兼容说明

特性 / 方法	支持版本	备注
<code>html_id</code> 属性	2.16.0+	用于直接操作 HTML 元素
<code>toggle</code> 参数 (<code>default_classes</code>)	2.7.0+	用于切换 CSS 类
<code>on()</code> 方法支持双处理器 (Python+JS)	2.18.0+	可同时指定 Python 回调和 JS 回调
<code>strict</code> 参数 (绑定相关方法)	3.0.0+	绑定属性时检查目标对象是否存在该属性

总结

`ui.scroll_area` 是 NiceGUI 中功能强大的滚动容器组件，核心优势在于：

- 1. 基于 Quasar 组件，滚动性能稳定，支持丰富的样式自定义；
- 2. 完善的事件监听和程序化控制，满足复杂交互需求（如同步滚动、滚动定位）；
- 3. 与 NiceGUI 其他组件（如 `ui.number` `ui.switch`）无缝集成，支持属性绑定和状态同步。

适用于长文本展示、多元素滚动列表、联动滚动面板等场景，通过 `classes` `props` 可灵活适配不同 UI 设计风格。